

Herramientas de testing en OpenMarkov

Informe de referencia

Manuel Arias Calleja — CISIAD, UNED

Abril 2026

Índice

1. Introducción	2
2. Herramientas preexistentes	2
2.1. JUnit 5	2
2.2. AssertJ Core	3
2.3. AssertJ Swing	4
2.4. JaCoCo	4
2.5. Maven Surefire	4
3. Herramientas incorporadas	4
3.1. jqwik — tests basados en propiedades	4
3.1.1. Anotaciones principales	5
3.1.2. Anotaciones de restricción	7
3.1.3. API de generadores (Arbitraries)	8
3.1.4. Fichero de semillas	9
3.2. PITest — testing de mutaciones	9
3.2.1. Configuración	9
3.2.2. Ejecución e informe	9
3.2.3. Interpretación de resultados	10
3.2.4. Ejemplo ilustrativo	10
4. Convenciones del proyecto	11
5. Inventario de tests existentes	11

1. Introducción

Este documento describe las herramientas de testing disponibles en OpenMarkov, tanto las que ya se usaban antes del esfuerzo actual de cobertura como las que se han incorporado recientemente. El objetivo es servir de referencia rápida para cualquier desarrollador que necesite escribir o interpretar tests en el proyecto.

La Tabla 1 resume el conjunto completo de herramientas.

Herramienta	Versión	Origen	Propósito
JUnit 5 (JUnit Platform + Jupiter)	5.13.0	Preexistente	Plataforma y anotaciones de tests
AssertJ Core	4.0.0-M1	Preexistente	Aserciones fluidas
AssertJ Swing	3.17.1	Preexistente	Tests de interfaz gráfica Swing
jqwik	1.9.2	Nuevo	Tests basados en propiedades
PITest (pitest-maven)	1.17.3	Nuevo	Testing de mutaciones
pitest-junit5-plugin	1.2.0	Nuevo	Integración PITest con JUnit 5
JaCoCo	0.8.11	Preexistente	Cobertura de código
Maven Surefire	3.5.2	Preexistente	Ejecución de tests en Maven

Tabla 1: Herramientas de testing en OpenMarkov

2. Herramientas preexistentes

2.1. JUnit 5

JUnit 5 es la plataforma de ejecución de tests del proyecto. Está gestionado mediante un BOM (*Bill of Materials*) en el POM padre, lo que garantiza versiones coherentes en todos los módulos.

La anotación fundamental es `@Test`, que marca un método como caso de prueba. JUnit se encarga de descubrir estos métodos, ejecutarlos y reportar los resultados.

```
1 import org.junit.jupiter.api.Test;
2 import static org.assertj.core.api.Assertions.*;
3
4 class VariableTest {
5
6     @Test
7     void numStatesMatchesConstructorArgument() {
8         Variable v = new Variable("Rain", 3);
9         assertThat(v.getNumStates()).isEqualTo(3);
10    }
```

11 }

Listado 1: Test básico con JUnit 5

Otras anotaciones de JUnit 5 usadas en el proyecto:

@BeforeEach

Método ejecutado antes de *cada* test. Se usa para inicializar el estado compartido (por ejemplo, crear una red **ProbNet** fresca).

@AfterEach

Método ejecutado después de cada test. Se usa para liberar recursos.

@Disabled

Desactiva un test temporalmente sin eliminarlo. Surefire lo contabiliza como *skipped*.

@Nested

Agrupar tests relacionados en clases internas, creando una jerarquía legible en los informes.

2.2. AssertJ Core

AssertJ proporciona una API fluida de aserciones que sustituye a los `assertEquals/assertTrue` clásicos de JUnit. Su principal ventaja es la legibilidad y los mensajes de error detallados.

```
1 // Comparación simple
2 assertEquals(node.getParents(), parentNode);
3
4 // Comparación numérica con tolerancia
5 assertEquals(sum, 1.0, within(1e-10));
6
7 // Verificar excepciones
8 assertEqualsThrownBy(edit::executeEdit)
9     .isInstanceOf(ConstraintViolatedException.class);
10
11 // Verificar identidad de objeto (misma referencia)
12 assertEquals(restored, original);
13
14 // Verificar contenido exacto de una colección
15 assertEquals(copy.getValues(), original.getValues());
```

Listado 2: Aserciones fluidas con AssertJ

Los métodos más usados en el proyecto son:

isEqualTo(expected)

Igualdad por `equals()`.

isSameAs(expected)

Identidad de referencia (`==`).

isNull() / isNotNull()

Verificar nulidad.

`contains(element)`

La colección contiene el elemento.

`containsExactly(elements...)`

Contenido y orden exactos.

`doesNotContain(element)`

Ausencia en una colección.

`isCloseTo(value, within(delta))`

Comparación numérica con tolerancia, imprescindible para valores de potenciales.

`as(format, args...)`

Añade un mensaje descriptivo que aparece en caso de fallo. Muy útil en bucles.

`assertThatThrownBy(callable)`

Verifica que una lambda lanza una excepción del tipo esperado.

2.3. AssertJ Swing

Librería para tests funcionales de interfaces Swing. Permite localizar componentes, simular clicks y teclas, y verificar el estado visual. Se usa en el módulo `org.openmarkov.gui` y en `org.openmarkov.integrationtests`.

2.4. JaCoCo

JaCoCo (*Java Code Coverage*) instrumenta el bytecode para medir qué líneas y ramas se ejecutan durante los tests. Se configura en el POM padre y genera informes HTML en `target/site/jacoco/index.html` de cada módulo.

El script `test.sh` del workspace ofrece la opción `-r` para abrir el informe de cobertura automáticamente tras ejecutar los tests.

2.5. Maven Surefire

Plugin de Maven que descubre y ejecuta los tests. Está configurado con la versión 3.5.2 y usa el motor de JUnit Platform, que permite ejecutar tanto tests de JUnit 5 como de jqwik en la misma pasada.

3. Herramientas incorporadas

3.1. jqwik — tests basados en propiedades

jqwik¹ es un motor de *property-based testing* para Java que se integra con JUnit Platform. En lugar de verificar un ejemplo concreto, una propiedad expresa un invariante que debe

¹<https://jqwik.net/>

cumplirse para *cualquier* entrada válida. jqwik genera automáticamente cientos de entradas aleatorias y, si encuentra un contraejemplo, lo reduce al caso mínimo que reproduce el fallo (*shrinking*).

3.1.1. Anotaciones principales

@Property

Marca un método como test de propiedad. A diferencia de `@Test`, que se ejecuta una sola vez, jqwik ejecuta un método `@Property` múltiples veces (por defecto 1000) con valores diferentes generados automáticamente para cada parámetro anotado con `@ForAll`.

La idea fundamental es expresar un **invariante**: una condición que debe cumplirse para *cualquier* entrada válida, no sólo para un ejemplo concreto. Si jqwik encuentra una combinación de valores que viola el invariante, reporta el contraejemplo y lo reduce automáticamente al caso mínimo que reproduce el fallo.

En el siguiente ejemplo, el invariante es: «el tamaño de la tabla de un potencial siempre es igual al producto del número de estados de todas sus variables». jqwik generará 1000 listas de variables con distintos números de estados y verificará que la propiedad se cumple en todos los casos.

```
1  @Property
2  void tableSizeIsProductOfAllStateCounts(
3      @ForAll("variableLists") List<Variable> variables) {
4      TablePotential tp = new TablePotential(
5          variables, PotentialRole.CONDITIONAL_PROBABILITY)
6          ;
7      int expected = variables.stream()
8          .mapToInt(Variable::getNumStates)
9          .reduce(1, (a, b) -> a * b);
10     assertThat(tp.getTableSize()).isEqualTo(expected);
11 }
```

Listado 3: `@Property`: invariante de tamaño de tabla

@ForAll

Se coloca delante de un parámetro de un método `@Property` para indicar a jqwik que debe generar un valor aleatorio para ese parámetro en cada ejecución.

Para tipos primitivos (`int`, `double`, `String`, etc.), jqwik sabe generar valores automáticamente. Se puede acotar el rango con anotaciones de restricción como `@IntRange` o `@DoubleRange`. Para tipos de dominio como `Variable` o `List<Variable>`, es necesario indicar un generador personalizado mediante `@ForAll("nombreDelGenerador")` (véase `@Provide`).

En el siguiente ejemplo, jqwik genera automáticamente valores de `numStates` entre 1 y 50 sin necesidad de un generador personalizado:

```
1  @Property
2  void numStatesMatchesIntConstructor(
3      @ForAll @IntRange(min = 1, max = 50) int numStates) {
```

```

4     Variable v = new Variable("X", numStates);
5     assertThat(v.getNumStates()).isEqualTo(numStates);
6 }

```

Listado 4: `@ForAll` con tipos primitivos y restricciones

Se pueden combinar múltiples parámetros `@ForAll` en una misma propiedad. jqwik genera todas las combinaciones:

```

1 @Property
2 void reorderSwap_preservesValues(
3     @ForAll @IntRange(min = 2, max = 5) int sA,
4     @ForAll @IntRange(min = 2, max = 5) int sB) {
5     Variable a = new Variable("A", sA);
6     Variable b = new Variable("B", sB);
7     // jqwik prueba con (sA=2, sB=2), (sA=3, sB=5), etc.
8     // ...
9 }

```

Listado 5: `@ForAll` con múltiples parámetros

`@Provide`

Marca un método que devuelve un `Arbitrary<T>` — es decir, un generador de valores aleatorios de tipo `T` (véase la Sección 3.1.3). Este generador se conecta a un parámetro `@ForAll` por nombre: si el parámetro dice `@ForAll("variableLists")`, jqwik buscará un método anotado con `@Provide` que se llame `variableLists()`.

IntelliJ marca estos métodos como no usados porque jqwik los invoca por reflexión; se recomienda añadir `@SuppressWarnings("unused")`.

El siguiente generador produce listas de 1 a 4 variables, cada una con entre 2 y 5 estados, con nombres únicos (`V0`, `V1`, ...):

```

1 @Provide
2 @SuppressWarnings("unused")
3 Arbitrary<List<Variable>> variableLists() {
4     return Arbitraries.integers().between(1, 4).flatMap(
5         numVars ->
6             Arbitraries.integers().between(2, 5)
7                 .list().ofSize(numVars)
8                 .map(stateCounts -> {
9                     List<Variable> vars = new ArrayList<>(numVars);
10                     for (int i = 0; i < numVars; i++) {
11                         vars.add(new Variable("V" + i,
12                                                 stateCounts.get(i))
13                     );
14                 }
15                 return vars;
16             })
17 );
18 }

```

Listado 6: `@Provide`: generador de listas de variables

El generador se usaría así en una propiedad:

```
1 @Property
2 void valuesLengthEqualsTableSize(
3     @ForAll("variableLists") List<Variable> variables) {
4     TablePotential tp = new TablePotential(
5         variables, PotentialRole.CONDITIONAL_PROBABILITY)
6     ;
7     assertThat(tp.getValues()).hasSize(tp.getTableSize());
8 }
```

Listado 7: Uso de un generador personalizado en una propiedad

@Example

Equivalente a `@Test` de JUnit, pero dentro de una clase gestionada por jqwik. Se usa para tests de regresión específicos que conviven con propiedades en la misma clase. A diferencia de `@Property`, un `@Example` se ejecuta exactamente una vez, con valores fijos elegidos por el desarrollador.

Es especialmente útil para documentar bugs concretos que se han descubierto: el test de regresión asegura que el bug no reaparezca. En el siguiente ejemplo, se verifica que hacer undo de un `AddLinkEdit` no muta la lista de variables del potencial original de un nodo `UTILITY`:

```
1 @Example
2 void undoDoesNotMutateOldUtilityPotentialVariables()
3     throws DoEditException {
4     ProbNet net = new ProbNet(
5         InfluenceDiagramType.getUniqueInstance());
6     net.getPNESupport().setWithUndo(true);
7     Variable vC = new Variable("C", 2);
8     Variable vU = new Variable("U"); // utility
9     new AddNodeEdit(net, vC, NodeType.CHANCE).executeEdit();
10    new AddNodeEdit(net, vU, NodeType.UTILITY).executeEdit();
11
12    AddLinkEdit edit = new AddLinkEdit(net, vC, vU, true);
13    edit.executeEdit();
14    edit.undo();
15
16    List<Variable> varsAfterUndo =
17        net.getNode(vU).getPotentials().getFirst()
18        .getVariables();
19    assertThat(varsAfterUndo).doesNotContain(vC);
20 }
```

Listado 8: `@Example`: test de regresión específico

3.1.2. Anotaciones de restricción

Estas anotaciones se combinan con `@ForAll` para acotar el rango de los valores generados:

`@IntRange(min, max)`

Restringe un `int` al rango `[min, max]`. Los valores por defecto son `min = 0`, `max = Integer.MAX_VALUE`, así que `@IntRange(max = 100)` es equivalente a `@IntRange(min = 0, max = 100)`.

`@DoubleRange(min, max)`

Restringe un `double` al rango especificado. Útil para factores de escala o probabilidades.

3.1.3. API de generadores (Arbitraries)

Un **generador** (tipo `Arbitrary<T>` en jqwik) es un objeto que sabe producir valores aleatorios de tipo `T`. Cuando jqwik ejecuta una propiedad, pide a cada generador que produzca un valor distinto en cada iteración, usando un generador de números pseudo-aleatorios interno. Además, si el test falla, jqwik usa el generador para *reducir* (*shrink*) el valor hacia el contraejemplo más pequeño posible — por ejemplo, reduciendo una lista de 5 elementos a una de 2 si con eso basta para reproducir el fallo.

Para tipos primitivos como `int` o `String`, jqwik proporciona generadores predefinidos. Para tipos de dominio como `Variable` o `List<Variable>`, hay que construir generadores personalizados componiendo los primitivos. La clase `Arbitraries` es el punto de entrada para esta composición.

Los combinadores más usados en el proyecto son:

`Arbitraries.integers().between(a, b)`

Genera enteros en el rango `[a, b]`, con distribución uniforme.

`Arbitraries.strings().alpha().ofMinLength(n).ofMaxLength(m)`

Genera cadenas compuestas sólo de letras, de longitud entre `n` y `m`.

`.flatMap(f)`

Encadena un generador con otro que *depende* del valor generado. Imprescindible cuando un valor determina la estructura de otro — por ejemplo, generar primero el número de variables y luego una lista de ese tamaño.

`.map(f)`

Transforma el valor generado sin cambiar la estructura del generador. Se usa para construir objetos de dominio (`Variable`, `State[]`) a partir de valores primitivos.

`.list().ofSize(n)`

Genera una lista de exactamente `n` elementos del tipo base.

El Listado 6 muestra un ejemplo completo de composición: `flatMap` genera primero el número de variables (1–4), luego `.list().ofSize(numVars)` genera una lista de enteros de ese tamaño (los números de estados), y finalmente `.map` transforma la lista de enteros en una lista de objetos `Variable`.

3.1.4. Fichero de semillas

jqwik usa internamente un generador de números pseudoaleatorios para producir los valores de cada ejecución. La **semilla** (*seed*) es el valor inicial de ese generador: dada la misma semilla, jqwik produce exactamente la misma secuencia de valores, lo que permite reproducir un fallo de forma determinista.

jqwik almacena las semillas de las ejecuciones anteriores en un fichero `.jqwik-database` en la raíz de cada módulo. Si un test falló en la ejecución anterior, jqwik reutiliza la misma semilla para verificar que el fallo se ha corregido antes de probar semillas nuevas.

Este fichero es local a cada máquina de desarrollo y está excluido del control de versiones mediante `.gitignore` en todos los módulos.

3.2. PITest — testing de mutaciones

PITest² es una herramienta de *mutation testing*. Introduce pequeñas modificaciones (*mutantes*) en el código fuente compilado — por ejemplo, cambiar un `+` por un `-`, invertir una condición, o sustituir un valor de retorno — y ejecuta los tests. Si un test falla, el mutante ha sido *eliminado* (*killed*); si todos los tests pasan, el mutante ha *sobrevivido*, lo que indica que los tests no detectan esa alteración del código.

3.2.1. Configuración

PITest está configurado en el `pluginManagement` del POM padre y activado con targets específicos en los módulos que lo usan (actualmente `org.openmarkov.core`):

```
1 <plugin>
2   <groupId>org.pitest</groupId>
3   <artifactId>pitest-maven</artifactId>
4   <configuration>
5     <targetClasses>
6       <param>org.openmarkov.core.*</param>
7     </targetClasses>
8     <targetTests>
9       <param>org.openmarkov.core.*</param>
10    </targetTests>
11  </configuration>
12 </plugin>
```

Listado 9: Configuración de PITest en el POM del módulo core

3.2.2. Ejecución e informe

```
1 # Ejecutar mutation testing en el módulo core
2 mvn -f org.openmarkov.core/pom.xml pitest:mutationCoverage
3
```

²<https://pitest.org/>

```

4 # El informe HTML se genera en:
5 # org.openmarkov.core/target/pit-reports/YYYYMMDDHHMI/index.html

```

Listado 10: Ejecución de PITest

3.2.3. Interpretación de resultados

PITest genera un informe HTML que muestra, para cada clase:

- El número de mutantes generados.
- Cuántos fueron eliminados (*killed*) por los tests.
- Cuántos sobrevivieron (*survived*) — indican debilidades en los tests.
- La *mutation score*: porcentaje de mutantes eliminados.

Un mutation score bajo no indica necesariamente que el código esté mal, sino que los tests no son suficientemente sensibles a ciertos cambios. Es una métrica complementaria a la cobertura de líneas de JaCoCo: un código puede tener 100 % de cobertura de líneas pero un mutation score bajo si los tests no verifican los valores correctos.

3.2.4. Ejemplo ilustrativo

Para entender cómo funciona PITest en la práctica, consideremos un método sencillo que calcula el número de estados de una variable discreta:

```

1 public int getNumStates() {
2     return states.size();
3 }

```

Listado 11: Método original que será mutado por PITest

Un test que simplemente invoque el método sin verificar el resultado no eliminará ningún mutante:

```

1 @Test
2 void testGetNumStates() {
3     Variable v = new Variable("X", 3);
4     v.getNumStates(); // no hay asercion!
5 }

```

Listado 12: Test débil — no elimina mutantes

PITest genera varios mutantes a partir del código original. Por ejemplo, el mutador `RETURN_VALS` sustituye el valor de retorno por 0:

```

1 // Mutante: replaced int return with 0
2 public int getNumStates() {
3     return 0; // <-- mutacion introducida
4 }

```

Listado 13: Mutante generado: valor de retorno reemplazado por 0

El test débil del Listado 12 no detecta este cambio, porque no comprueba el valor devuelto — el mutante *sobrevive*. Un test que sí verifica el resultado elimina al mutante:

```
1 @Test
2 void testGetNumStates() {
3     Variable v = new Variable("X", 3);
4     assertThat(v.getNumStates()).isEqualTo(3);
5 }
```

Listado 14: Test robusto — elimina el mutante

Con esta aserción, cuando PITest sustituye `states.size()` por 0, el test falla y el mutante queda eliminado (*killed*). Otros mutadores típicos de PITest incluyen:

- **CONDITIONALS_BOUNDARY**: cambia `<` por `<=` (y viceversa).
- **NEGATE_CONDITIONALS**: invierte condiciones (`==` $\rightarrow$ `!=`).
- **MATH**: sustituye operadores aritméticos (`+` $\rightarrow$ `-`, `*` $\rightarrow$ `/`).
- **VOID_METHOD_CALLS**: elimina llamadas a métodos `void`.
- **EMPTY_RETURNS**: devuelve colecciones vacías o cadenas vacías en lugar del valor real.

4. Convenciones del proyecto

- Las clases de test se nombran `*Test` para tests de ejemplo y `*PropertyTest` para tests basados en propiedades.
- Los tests de propiedades conviven con tests `@Example` en la misma clase cuando el ejemplo es una regresión del mismo componente.
- Los proveedores (`@Provide`) se agrupan al final de la clase en una sección separada por comentarios.
- Se usa `@SuppressWarnings("unused")` en los métodos `@Provide` para evitar advertencias de IntelliJ.
- Se declara `throws DoEditException` en las propiedades que ejecutan edits, en lugar de capturar la excepción.
- Los helpers de creación de redes (`freshBayesianNet()`, `twoNodeBN()`) se definen como métodos privados o records al inicio de la clase de test.

5. Inventario de tests existentes

La Tabla 2 lista las clases de test basadas en propiedades añadidas hasta la fecha, todas en el módulo `org.openmarkov.core`.

Clase de test	Props.	Qué verifica
VariablePropertyTest	12	Invariantes de nombre/-baseName/timeSlice, índice de estados, constructor copia, potenciales delta
TablePotentialPropertyTest	11	Tamaño de tabla, offsets, scalePotential , constructor copia, setUniform
TablePotentialReorderProjectPropertyTest	10	reorder() preserva valores para 2 y 3 variables, identidad, round-trip; tableProject() con evidencia parcial, total y nula
AddNodeEditPropertyTest	9	Ciclo do/undo/redo de AddNodeEdit , rechazo de nombres duplicados
AddLinkEditPropertyTest	11	Ciclo do/undo/redo de AddLinkEdit , rechazo de self-loop y links duplicados, regresión de mutación de potencial UTILITY
ProbNetOperationsBarrenNodesPropertyTest	12	removeBarrenNodes() : seguridad de nodos de interés/evidencia, eliminación de hojas/cadenas/-diamantes, idempotencia, completitud
Total	65	

Tabla 2: Clases de test basadas en propiedades (jqwik)